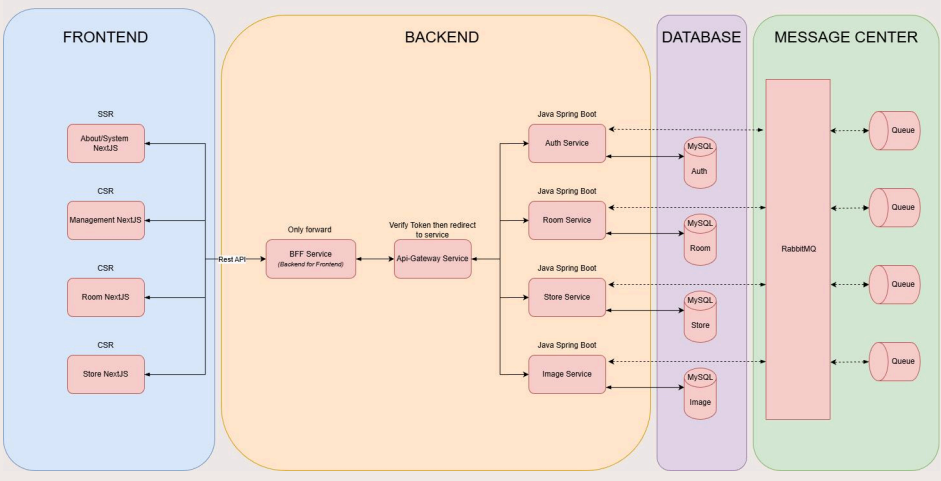
J2N Projects

J2N Projects is a full-stack project designed with a micro-frontend architecture using Turborepo for modular UI development. The backend is structured as a microservices ecosystem to ensure scalability and service isolation, with MySQL serving as the primary database for data storage and management.



Frontend Layer *(Micro Frontends using Next.js)*

The UI layer is split into multiple independent Next.js applications, where each app is responsible for a specific business domain:

Application	Render Mode	Purpose
About/System	Server side render	System introduction and public info pages
Management	Client side render	Internal management dashboard
Room	Client side render	Room management features
Store	Client side render	Store, product, and order handling

Key Characteristics
- Each frontend is deployed independently.
- Failures in one module do not affect the others.
- All frontends communicate with backend exclusively through the BFF Service, ensuring consistent data shape and shielding the UI from backend complexity.

UI Design: https://www.figma.com/design/oVSfPR7ScJOFYduTKxa8OA/J2N-Design-UI?node-id=2111-2&p=f&t=fhxeHcrVHzC4kNzV-0

Backend Layer

The backend consists of two main parts: the BFF, the API Gateway, and a set of domain-driven microservices.

(a) BFF Service (Backend for Frontend)
- Acts as an adapter between all frontend apps and the internal backend system.
- Responsibilities:
  - Normalize requests and responses
  - Perform lightweight validation
  - Aggregate data from multiple services (future support)
  - Prevent FE from calling internal services directly

The BFF contains no business logic; it mainly forwards requests to the API Gateway.

(b) API Gateway Service
The API Gateway is the single controlled entry point to the backend.
Key responsibilities:
- Verify JWT tokens
- Route requests to the correct microservice
- Apply rate-limiting, throttling, and logging (expandable)
- Hide internal microservices from the client

(c) Microservices Layer (Java Spring Boot)
Each service is a standalone Spring Boot application following the "one service, one domain" approach:

Service	Responsibility
Auth Service	Authentication, registration, login, token generation and management users
Room Service	Management Room info, status, reservations and caculate bills
Store Service	Management Products, orders, inventory
Image Service	Image upload, processing, metadata

Database Layer *(MySQL per Service)*

Each microservice has its own dedicated MySQL database.

Advantages
- No shared schema → loose coupling
- Each service manages its own schema (clean domain boundaries)
- Easy to migrate or scale only the services that need it
- System stability is improved (failures in one DB do not affect others)

Message Layer *(RabbitMQ)*

RabbitMQ is used for asynchronous communication between services.

Use Cases
- Sending notifications or emails
- Cross-service data synchronization
- Background/long-running tasks
- High-load event processing

Each type of event has its own queue to maintain isolation and reliability.

Request Flow & Event Flow

I. Request Flow (Frontend → Backend)
1. Frontend sends a request to the BFF Service.
2. BFF normalizes and forwards the request to the API Gateway.
3. API Gateway:
   - Validates JWT
   - Routes the request to the appropriate microservice
4. Microservice processes business logic and interacts with its MySQL database.
5. Response returns through the same chain:
   Service → Gateway → BFF → Frontend

II. Event Flow (Asynchronous Processing)
1. A microservice triggers an event (e.g., user created, new order, image uploaded).
2. The service publishes the event to a specific RabbitMQ queue.
3. Other subscribed services receive and process the event.
4. Tasks run in the background without delaying the user-facing response.

This ensures smoother performance and better scalability.

Security & Authentication

I. JWT Authentication
- User logs in from the frontend → BFF → Auth Service.
- Auth Service returns a JWT token.
- Frontend stores the token securely.
- All future requests include this token through BFF → Gateway.

II. API Gateway Security
- Single point responsible for token verification.
- Internal services never receive unauthenticated direct requests.
- Optional IP whitelisting for service-to-service communication.

III. Microservice Isolation
- Each service accesses only its own database.
- Prevents cross-service data leakage or accidental coupling.

IV. RabbitMQ Security
- Only authorized services may publish or consume events.
- Ensures safe asynchronous communication.